

DEAP 1



DISTRIBUTED
EVOLUTIONARY
ALGORITHMS IN
PYTHON

Developed by a team at Université Laval in **Quebec**, Canada.

The key contributors include Francois-Michel De Rainville, Marc-André Gardner, Christian Gagné, and Marc Parizeau.

LTU CS CJ Chung

Distributed Evolutionary Algorithms in Python

- A library for creating and testing evolutionary algorithms, including genetic algorithms (GAs), genetic programming (GP), evolution strategies (ES), and more.
- DEAP provides tools for creating flexible, modular, and efficient algorithms to solve optimization problems in Python.
- Using any imaginable representation such as List, Array, Set, Dictionary, Tree, Numpy Array, etc.

Distributed Evolutionary Algorithms in Python

■ Links

- <https://github.com/DEAP>
- <https://pypi.org/project/deap/>

■ YouTube Video:

- “A Gentle Introduction to Genetic Algorithms with Python and DEAP”
- <https://youtu.be/6h5Imly3kpA?si=vwuo-Wysq29PSsHC> (1hr 35 min)

Documentation



Project Homepage » DEAP 1.4.1 documentation »

Docs for other versions

DEAP 1.0 (Stable)
DEAP 0.9

Other resources

Downloads
Issues
Mailing List
Twitter

DEAP documentation

DEAP is a novel evolutionary computation framework for rapid prototyping and testing of ideas. It seeks to make algorithms explicit and data structures transparent. It works in perfect harmony with parallelisation mechanism such as multiprocessing and [SCOOP](#). The following documentation presents the key concepts and many features to build your own evolutions.



DISTRIBUTED
EVOLUTIONARY
ALGORITHMS IN
PYTHON

- **First steps:**

- [Overview \(Start Here!\)](#)
- [Installation](#)
- [Porting Guide](#)

- **Basic tutorials:**

- [Part 1: creating types](#)
- [Part 2: operators and algorithms](#)
- [Part 3: logging statistics](#)
- [Part 4: using multiple processors](#)

- **Advanced tutorials:**

- [Genetic Programming](#)
- [Checkpointing](#)
- [Constraint Handling](#)
- [Benchmarking Against the Bests \(BBOB\)](#)
- [Inheriting from Numpy](#)

- [Examples](#)

- [Library Reference](#)
- [Release Highlights](#)
- [Contributing](#)
- [About DEAP](#)

Getting Help

Having trouble? We'd like to help!

- Search for information in the archives of the [deap-users mailing list](#), or post a question.
- Report bugs with DEAP in our [issue tracker](#).

<https://deap.readthedocs.io/en/master/index.html>

Key Features of DEAP (1 /2)

- Flexible Structure: highly modular, allowing users to define and customize the components of an EA, such as individuals, populations, selection methods, crossover and mutation operators, and evaluation functions.
- Object-Oriented Design:
 - **Creator:** Used to define problem-specific objects like
 - individuals,
 - fitness values, and
 - types of optimization (e.g., maximization or minimization).
 - **Toolbox:** Manages functions related to the genetic algorithm, such as
 - selection,
 - mutation, and
 - evaluation functions.

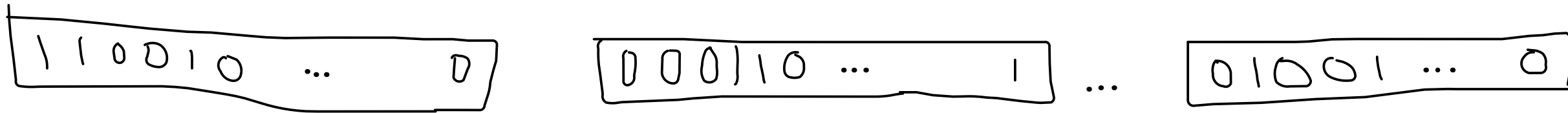
Key Features of DEAP (2 /2)

- Supports Multiple Evolutionary Algorithms such as:
 - ES,
 - GP,
 - PSO (Particle swarm optimization)
- Pre-built Algorithms:
 - eaSimple,
 - eaMuPlusLambda, and
 - eaMuCommaLambda
- allows parallel execution of evaluations across multiple CPU cores or on distributed systems, improving speed for computationally intensive tasks

Basic Workflow in DEAP

1. **Define Fitness (evaluation) Function:** Define the objective function to be maximized or minimized
2. **Create Individual & Population:** Define the individual structure (e.g., lists, arrays) and create a population of these individuals
3. **Register Evolutionary Operators** like selection, crossover, mutation, and evaluation within the *toolbox*
4. **Run Algorithm:** Use one of DEAP's pre-built algorithms or write a custom loop to evolve the population over generations
5. **Retrieve Results:** After running the algorithm, extract the best individual or analyze the evolved population

Example 1: **DEAP_maxOne.ipynb** on Canvas




```
!pip install deap --quiet
```

DEAP_maxOne.ipynb on Canvas

```
import random
from deap import base, creator, tools, algorithms

# Define the fitness class (single objective maximization)
creator.create("FitnessMax", base.Fitness, weights=(1.0,)) # if -1.0, minimization
creator.create("Individual", list, fitness=creator.FitnessMax)

# Define the evaluation function
def evalOneMax(indv):
    return sum(indv),

# Create the toolbox
toolbox = base.Toolbox()
toolbox.register("attr_bool", random.randint, 0, 1)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool, n=25)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
toolbox.register("evaluate", evalOneMax)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3) # see EA_intro.pdf
```

DEAP_maxOne.ipynb on Canvas

```
# Run the algorithm
population = toolbox.population(n=300)
for gen in range(30):
    # varAnd: apply both variation operators—crossover and mutation—to a population of individuals
    offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)
    fitnesses = map(toolbox.evaluate, offspring)
    for ind, fit in zip(offspring, fitnesses):
        ind.fitness.values = fit
    population = toolbox.select(offspring, k=len(population))

# Get the best individual
best_ind = tools.selBest(population, k=4)
print("Best 4 individuals:", best_ind)

best_ind = tools.selBest(population, k=1)[0]
print("Best individual:", best_ind)
print("Best fitness:", best_ind.fitness.values[0])
```

Best 4 individuals: [[1, 1], [1, 1, ...
 Best individual: [1, 1]
 Best fitness: 25.0

4

Python Review



The map() function in Python applies a given function to all items in an iterable
(like a list or tuple) and returns an iterator with the results.

```
def square(x):  
    return x * x
```

List of numbers

```
numbers = [1, 2, 3, 4, 5]
```

Use map to apply 'square' to each element in 'numbers'

```
squared_numbers = map(square, numbers)
```

Convert the map object to a list to see the results

```
print(list(squared_numbers)) # Output: [1, 4, 9, 16, 25]
```



```
[1, 4, 9, 16, 25]
```



using lamda function

```
numbers = [1, 2, 3, 4, 5]
```

```
squared_numbers = map(lambda x: x * x, numbers)
```

```
print(list(squared_numbers)) # Output: [1, 4, 9, 16, 25]
```



```
[1, 4, 9, 16, 25]
```

Python Review



```
# zip() combines multiple iterables such as lists, tuples, strings, dict etc,  
# into a single iterator of tuples
```

```
names = ['John', 'Alice', 'Bob', 'Lucy']  
scores = [85, 90, 78, 92]
```

```
res = zip(names, scores) # res is an object  
print(list(res))
```



```
[('John', 85), ('Alice', 90), ('Bob', 78), ('Lucy', 92)]
```

Example 2: Function Optimization

■ DEAP_funcOp1.ipynb on Canvas

```
def sphere(x):
    return x[0]**2 + x[1]**2,

def polynomial(x): # minima at y=0
    return (x[0] ** 3 - 2 * x[0] ** 2 - x[0] + 2) ** 2,

def rosenbrock(x):
    return (1-x[0])**2 + 100*(x[1]-x[0]**2)**2,

def booth(x): # minima at (1,3)
    return ((x[0] + 2 * x[1] - 7) ** 2 + (2 * x[0] + x[1] - 5) ** 2, )

def ackley(x): # minima at (0,0)
    a = 20
    b = 0.2
    c = 2 * np.pi
    d = 2
    sum1 = -a * np.exp(-b * np.sqrt((1 / d) * (x[0] ** 2 + x[1] ** 2)))
    sum2 = -np.exp((1 / d) * (np.cos(c * x[0]) + np.cos(c * x[1])))
    return (sum1 + sum2 + a + np.exp(1),) # return as a tuple
```

What is the trailing comma?

A comma after the return statement in Python

```
def no_trailing_comma(x):  
    return x + [10]  
  
def trailing_comma(x):  
    return x + [10],  
  
data = [1, 2, 3]  
  
no_comma_value = no_trailing_comma(data)  
comma_value = trailing_comma(data)  
  
print(f"{no_comma_value} The return type is {type(no_comma_value)}")  
print(f"{comma_value} The return type is {type(comma_value)}")  
  
[1, 2, 3, 10] The return type is <class 'list'>  
([1, 2, 3, 10],) The return type is <class 'tuple'>
```

DEAP_funcOp1.ipynb

```
def deap_run(eval_func, numVar):
    # Create the toolbox
    toolbox = base.Toolbox()
    toolbox.register("attr_float", random.uniform, -5, 5)
    toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, n=numVar)
    toolbox.register("population", tools.initRepeat, list, toolbox.individual)
    toolbox.register("evaluate", eval_func)
    if numVar > 1:
        toolbox.register("mate", tools.cxTwoPoint) # Use cxTwoPoint for numVar > 1
        toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
    else:
        toolbox.register("mate", tools.cxUniform, indpb=0.5) # added this for single gene
        #toolbox.register("mate", tools.cxOnePoint) # this also works
        # Use a suitable operator for numVar = 1, e.g., mutGaussian
        toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.1)
    toolbox.register("select", tools.selTournament, tournsize=3)
```

DEAP_funcOp1.ipynb

```
# Run the algorithm
successes = 0
trials = 3
for tr in range(trials):
    population = toolbox.population(n=300)
    for gen in range(500):
        # varAnd: apply both variation operators—crossover and mutation—to a population of individuals
        offspring = algorithms.varAnd(population, toolbox, cxpb=0.5, mutpb=0.1)
        fitnesses = map(toolbox.evaluate, offspring) # see below for map()
        for ind, fit in zip(offspring, fitnesses):
            ind.fitness.values = fit
        population = toolbox.select(offspring, k=len(population))

# Get the best individual
best_ind = tools.selBest(population, k=1)[0]
print(f"----- Trial: {tr} ")
print("Best individual:", best_ind)
if best_ind.fitness.values[0] < 0.00005:
    successes += 1
    print("Best fitness:", best_ind.fitness.values[0])
print(f"\nSystem Success %: {successes*100/trials}")
```



```
##### M A I N #####
print("\n***** Sphere Function *****")
deap_run(sphere, 2)

print("\n***** Polynomial Function *****")
deap_run(polynomial, 1)

print("\n***** Rosenbrock Function *****")
deap_run(rosenbrock, 2)
```

```
***** Sphere Function *****
----- Trial: 0
Best individual: [0.0, 0.0]
Best fitness: 0.0
----- Trial: 1
Best individual: [0.0, 0.0]
Best fitness: 0.0
----- Trial: 2
Best individual: [0.0, 0.0]
Best fitness: 0.0
```

System Success %: 100.0

```
***** Polynomial Function *****
----- Trial: 0
Best individual: [-1.000660481863764]
Best fitness: 1.5721801137011518e-05
----- Trial: 1
Best individual: [1.999264165587377]
Best fitness: 4.863514931891199e-06
----- Trial: 2
Best individual: [0.9988675278589487]
Best fitness: 5.135777213995181e-06
```

System Success %: 100.0

```
***** Rosenbrock Function *****
----- Trial: 0
Best individual: [1.4414566287897674, 2.112232064682278]
Best fitness: 0.31345985833079726
----- Trial: 1
Best individual: [1.2327010010444521, 1.518654450795153]
Best fitness: 0.054230271904769055
----- Trial: 2
Best individual: [1.1499663311228296, 1.3036501356204244]
Best fitness: 0.0577303023766932
```

System Success %: 0.0

Example 3: **DEAP_maxSum.ipynb** using `algorithms.eaSimple()`

```
# Step 1: Define the evaluation function
def eval_individual(individual):
    return sum(individual), # The comma makes it a tuple, which DEAP expects

# Step 2: Set up DEAP components
# Create a fitness function to maximize
creator.create("FitnessMax", base.Fitness, weights=(1.0,))
# Create the individual, which is a list of integers
creator.create("Individual", list, fitness=creator.FitnessMax)

# Initialize the toolbox
toolbox = base.Toolbox()

# Register the attributes for each gene (random integers between 0 and 100) inclusive
toolbox.register("attr_int", random.randint, 0, 100)
# Create an individual with 10 attributes
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_int, 10)
# Create a population of 50 individuals
toolbox.register("population", tools.initRepeat, list, toolbox.individual, 50)

# Register the evaluation, crossover, mutation, and selection functions
toolbox.register("evaluate", eval_individual)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)
```

Step 3: Set up the main function to run the algorithm

```
def main():
    # Initialize population
    population = toolbox.population()

    # Run the algorithm
    final_population = algorithms.eaSimple(
        population,
        toolbox,
        cxpb=0.5, # Crossover probability
        mutpb=0.2, # Mutation probability
        ngen=40,   # Number of generations
        verbose=False,
    )

    # Gather all individuals in the final population
    print(final_population)

    print("Final population:")
    for individual in final_population[0]:
        print(individual)
    top_individual = tools.selBest(final_population[0], 1)[0]
    print("Best individual:", top_individual)
    print("Fitness:", top_individual.fitness.values[0])

main()
```

Result

```
Final population:
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 0, 71, 77, 99, 83, 0, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
[78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
Best individual: [78, 57, 94, 94, 71, 77, 99, 83, 75, 100]
Fitness: 828.0
```

Example 4: DEAP_funcOp2.ipynb

- Func Optim using algorithms.eaSimple()
- Instead of varAnd() in DEAP_funcOp1.ipynb

DEAP_funcOp2.ipynb

```
# Run the algorithm
successes = 0
trials = 3
for tr in range(trials):
    population = toolbox.population(n=300)
    # Run the algorithm
    final_population = algorithms.eaSimple(
        population,
        toolbox,
        cxpb=0.5, # Crossover probability
        mutpb=0.2, # Mutation probability
        ngen=50,   # Number of generations
        verbose=False,
    )

    # Get the best individual
    best_ind = tools.selBest(final_population[0], 1)[0]
    print(f"----- Trial: {tr} ")
    print("Best individual:", best_ind)
    if best_ind.fitness.values[0] < 0.00005:
        successes += 1
    print("Best fitness:", best_ind.fitness.values[0])
print(f"\nSystem Success %: {successes*100/trials}")
```

```
##### M A I N #####
print("\n***** Sphere Function *****")
deap_run(sphere, 2)

print("\n***** Polynomial Function *****")
deap_run(polynomial, 1)

print("\n***** Rosenbrock Function *****")
deap_run(rosenbrock, 2)
```

```
***** Sphere Function *****
----- Trial: 0
Best individual: [0.0, 0.0]
Best fitness: 0.0
----- Trial: 1
Best individual: [0.0, 0.0]
Best fitness: 0.0
----- Trial: 2
Best individual: [0.0, 0.0]
Best fitness: 0.0

System Success %: 100.0
```

```
***** Polynomial Function *****
----- Trial: 0
Best individual: [1.0020631807098592]
Best fitness: 1.6991674795981774e-05
----- Trial: 1
Best individual: [0.9992323635168296]
Best fitness: 2.358871406555026e-06
----- Trial: 2
Best individual: [1.9992975566743523]
Best fitness: 4.432526484933419e-06

System Success %: 100.0
```

```
***** Rosenbrock Function *****
----- Trial: 0
Best individual: [0.9925587021931541, 1.0]
Best fitness: 0.022040026214492414
----- Trial: 1
Best individual: [0.904890738536837, 0.8048567814826688]
Best fitness: 0.02856316701445994
----- Trial: 2
Best individual: [0.7572397511755629, 0.5587715279694443]
Best fitness: 0.08036699988759956

System Success %: 0.0
```

Define bounds for each variable

- So far all the variables had same range. For example:

```
# Create the toolbox
toolbox = base.Toolbox()
toolbox.register("attr_float", random.uniform, -5, 5)
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_float, n=numVar)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```

Sample 5: DEAP_bounds_ES.ipynb

```
# Define bounds for each variable as (min, max)
BOUNDS_LOW = [-10, -5, -2] # Minimum bounds for variables
BOUNDS_HIGH = [10, 5, 10] # Maximum bounds for variables

# Create a fitness and individual class
creator.create("FitnessMin", base.Fitness, weights=(-1.0,)) # Minimize objective
creator.create("Individual", list, fitness=creator.FitnessMin)

# Initialize DEAP toolbox
toolbox = base.Toolbox()

# Attribute generator that initializes within the variable bounds
def create_individual():
    return [random.uniform(low, high) for low, high in zip(BOUNDS_LOW, BOUNDS_HIGH)]

# Register individual and population creation functions
toolbox.register("individual", tools.initIterate, creator.Individual, create_individual)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)
```



```

# Define an objective function
def evaluate(individual):
    # Example: Sphere function (just a simple sum of squares)
    return sum(x**2 for x in individual),

toolbox.register("evaluate", evaluate)

# Crossover and mutation operators
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selBest)

# Ensure that mutated individuals stay within bounds
def check_bounds(individual):
    for i in range(len(individual)):
        individual[i] = np.clip(individual[i], BOUNDS_LOW[i], BOUNDS_HIGH[i])
    return individual, 0 # Return individual and penalty (0 in this case, since repaired)

# Decorate the mutation operator to enforce bounds
# toolbox.decorate("mutate", tools.DeltaPenalty(lambda ind: min(ind) >= min(BOUNDS_LOW) and max(ind) <= max(BOUNDS_HIGH), 0))
toolbox.decorate("mutate", tools.DeltaPenalty(check_bounds, 0)) # Use check_bounds function

```

```
def printBest(popul, s):
    # Get the best individual
    best_ind = tools.selBest(popul[0], 1)[0]
    print(f"{s} Strategy --- \nBest individual: {best_ind} \
          \nBest fitness: {best_ind.fitness.values[0]}")

pop_size = 20 # Population size|

# Run the evolutionary algorithm
def esPlus():
    population = toolbox.population(n=pop_size)
    final_population = algorithms.eaMuPlusLambda(
        population,
        toolbox,
        mu=pop_size,
        lambda_=50,
        cxpb=0.7,
        mutpb=0.2,
        ngen=20,
        verbose=False
    )
    printBest(final_population, "Plus ")
```

$ES(\mu + \lambda)$

```
def esComma():
    population = toolbox.population(n=pop_size)
    final_population = algorithms.eaMuCommaLambda(
        population,
        toolbox,
        mu=pop_size,
        lambda_=50,
        cxpb=0.7,
        mutpb=0.2,
        ngen=500,
        verbose=False
    )
    printBest(final_population, "Comma")

if __name__ == "__main__":
    esPlus()
    esComma()
```

$ES(\mu, \lambda)$

```
Plus Strategy ---
Best individual: [np.float64(-0.0011444467736648838), np.float64(0.005812373744431281), np.float64(0.0013412641142217239)]
Best fitness: 3.689243638680507e-05
Comma Strategy ---
Best individual: [np.float64(-1.437695820577524e-11), np.float64(4.7527883672509977e-14), np.float64(-2.6933060674302e-12)]
Best fitness: 2.1395308372319073e-22
```

How to handle discrete values to mutate or Xover using DEAP?

- How about a solution vector with mixed variable types (continuous and discrete)?
- See next lecture, DEAP 2